REDIS ROADMAP · PHASE 6

Agentic AI Redis

Giving AI agents a durable, shared brain: memory, state, checkpoints

An end-to-end, self-contained handbook. Read it top to bottom and you'll understand every piece of Phase 6 — the four things Redis gives an agentic system, and the six-agent LangGraph-style project that ties them together.

What's inside

1. Why AI agents need Redis (the four pillars)
2. Concept 1 — Agent Memory (short-term / conversation)
3. Concept 2 — State Management
4. Concept 3 — Checkpointing (resumable runs)
5. Concept 4 — Shared Memory (blackboard, locks, pub/sub)
6. Project 5 — The Multi-Agent System (6 agents)
7. Running everything & mapping to real LangGraph
8. Production checklist & key takeaways

Prerequisite: Phases 1–5 complete · Runs fully offline (no API key required)

CHAPTER 1

Why AI Agents Need Redis

In Phase 5 you cached the *results* of AI work. Phase 6 is different: Redis becomes the **brain and nervous system** of an agent. The reason comes down to one uncomfortable fact.

> **THE FACT EVERYTHING FOLLOWS FROM**
>
> An LLM is **stateless**. Every API call is a blank slate — the model remembers nothing from the last call. So an "agent with memory" is really an agent whose *framework* stores everything in a fast, durable place and replays it. That place is Redis.

An agentic system needs four things, and Redis provides all four:

Pillar	What it is	Redis structure
Memory	The conversation log (what was said)	List
State	The task's working data (what it's doing)	Hash
Checkpoints	Snapshots so a run can resume after a crash	List (append-only)
Shared memory	A blackboard many agents read/write	Hash + Pub/Sub + locks

Memory vs. State — don't confuse them

This trips up almost everyone. **Memory** is the *conversation*. **State** is the *task's working data*. For a research agent solving "Write a report on Redis":

```
memory : the chat between the user and the agent
state  : { plan:[...], findings:[...], step:3, draft:"..." }
```

The rest of this handbook builds each pillar as a runnable example, then combines them into a six-agent system with a Planner, Researcher, Retriever, Tool Router, Executor, and Reviewer.

CHAPTER 2 · CONCEPT 1

Agent Memory

file `concepts/01_agent_memory.py`

The problem

If a user says "My name is Rahim" and later asks "What is my name?", the stateless model has no idea — unless you send the earlier turns back to it. So we store the conversation in Redis and replay recent turns into every new prompt.

```
user message → append to Redis LIST (conv:{id})
    → read last N turns → build prompt → LLM → reply
              → append reply to the SAME list
```

Why a Redis list is the perfect fit

A conversation is an *ordered* sequence of turns, and lists give us exactly the operations we need:

Command	Use
`RPUSH`	Append a new turn to the end — O(1)
`LRANGE`	Read a slice, e.g. the last 10 turns
`LTRIM`	Keep only the most recent N (windowing)
`EXPIRE`	Auto-forget conversations that go idle (TTL)

```
def add_message(conversation_id, role, content, max_turns=20, ttl=3600):
    key = f"conv:{conversation_id}:messages"
    turn = json.dumps({"role": role, "content": content})
    r.rpush(key, turn)              # newest at the tail
    r.ltrim(key, -max_turns, -1)    # sliding window: keep last N
    r.expire(key, ttl)              # forget idle conversations
```

WINDOWING: WHY WE DON'T KEEP EVERYTHING

LLMs have a limited context window and you pay per token, so we keep only the last N turns. For long chats you go one level up: keep the last N turns *verbatim* plus a rolling **summary** of older turns in a separate key, and build the prompt as `summary + recent turns`.

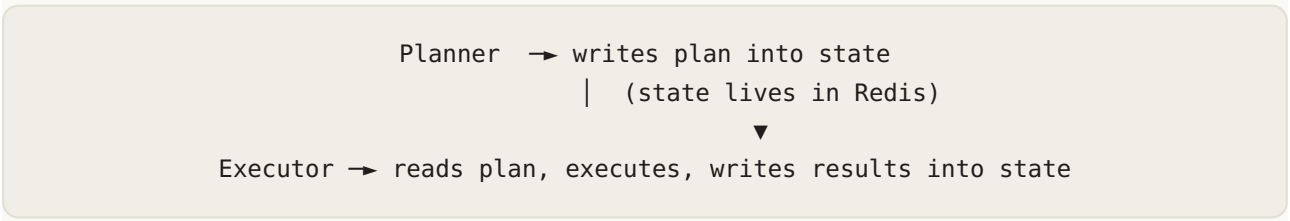
CHAPTER 3 · CONCEPT 2

State Management

file `concepts/02_state_management.py`

The shared "working memory"

In frameworks like LangGraph, a single **state** object is the source of truth that flows from one step (node) to the next. Each agent *reads* the state, does its job, and *writes* an updated state back. We store it as a Redis hash keyed by `run_id`.



Why put state in Redis instead of a Python variable?

- **Durability** — if the process crashes, the state survives (Chapter 4).
- **Sharing** — many agents/workers/machines read the same state.
- **Visibility** — you can inspect a running task from `redis-cli`.
- **Scaling** — steps can run on different servers and still coordinate.

```
def update_state(run_id, **changes):        # each agent returns deltas
    key = f"run:{run_id}:state"
    mapping = {f: json.dumps(v) for f, v in changes.items()}
    r.hset(key, mapping=mapping)            # update only these fields
    return get_state(run_id)
```

> **THE DECOUPLING THAT MAKES IT WORK**
>
> The Planner and Executor never call each other directly — they communicate *only* through shared state in Redis. That decoupling is what lets agent steps run independently, retry on failure, and scale horizontally.

CHAPTER 4 · CONCEPT 3

Checkpointing

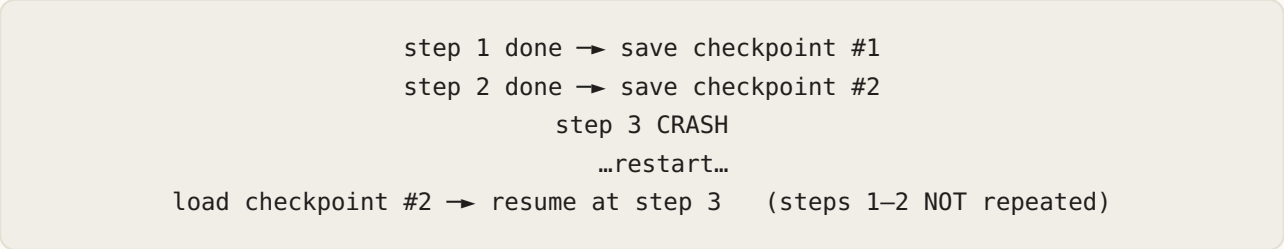
file `concepts/03_checkpointing.py`

Why long agent runs need it

Agent tasks are long and fragile: 20 LLM calls, external tool calls, minutes of runtime — any of which can time out, hit a rate limit, or crash. If step 18 of 20 dies and you must restart from step 1, you waste the time and money of everything you already did.

The solution: snapshot after every step

After each step, save a snapshot of the full state to Redis. On restart, load the latest snapshot and resume from exactly where it stopped.




```
def run(thread_id, ...):
    latest = load_latest(thread_id)         # any prior checkpoint?
    if latest:
        state = latest["state"]
        done  = set(state["completed_steps"])   # skip finished steps
    for step in STEPS:
        if step in done: continue           # already done -> skip
        state = do_work(step, state)        # the expensive part
        save_checkpoint(thread_id, step, state)  # persist immediately
```

KEEP THE WHOLE HISTORY, NOT JUST THE LATEST

Storing every checkpoint in an append-only list gives you three things: **crash recovery** (resume from latest), **time-travel debugging** (inspect any past step), and the ability to **fork** a run from an earlier point. This is exactly how LangGraph's checkpointer behaves — and LangGraph ships a Redis one for production.

CHAPTER 5 · CONCEPT 4

Shared Memory

file `concepts/04_shared_memory.py`

The blackboard pattern

When several specialist agents work on one task, they need to share what they discover. The classic solution is a **blackboard**: a shared space every agent can read from and write to. A Redis hash is a natural blackboard — fast, central, and visible to all.

```
        Researcher —writes→ [ BLACKBOARD in Redis ] ←reads— Reviewer
        Retriever  —writes→ [                     ] ←reads— Executor
```

This concept teaches three tools that make multi-agent cooperation reliable:

1. Shared state (the blackboard)

```
r.hset("run:007:blackboard", "Researcher", json.dumps(finding))
findings = r.hgetall("run:007:blackboard")   # any agent reads all
```

2. Distributed locks (no lost updates)

When two agents might write the same slot at once, a lock lets only one in at a time. The standard safe pattern is `SET key token NX PX ttl`: `NX` only sets if unlocked, `PX` auto-expires so a crashed holder can't lock forever, and a random token proves ownership on release (via a small atomic Lua script).

```
token = r.set(f"lock:{res}", uuid4(), nx=True, px=3000)  # try to lock
# ... critical section: only one agent here at a time ...
# release only if our token still matches (atomic Lua check-and-delete)
```

3. Pub/Sub (event-driven coordination)

Instead of agents constantly polling "are you done yet?", an agent `PUBLISH`es an event when it finishes, and interested agents `SUBSCRIBE` and react instantly.

```
r.publish("run:007:events", "Researcher_done")
# a subscribed Reviewer process starts the moment it sees this
```

> **THE TRIO, SUMMARIZED**
>
> Blackboard = *what agents know*. Locks = *safe concurrent writes*. Pub/Sub = *event-driven coordination*. Together they let many specialists cooperate without stepping on each other.

CHAPTER 6 · PROJECT 5

The Multi-Agent System

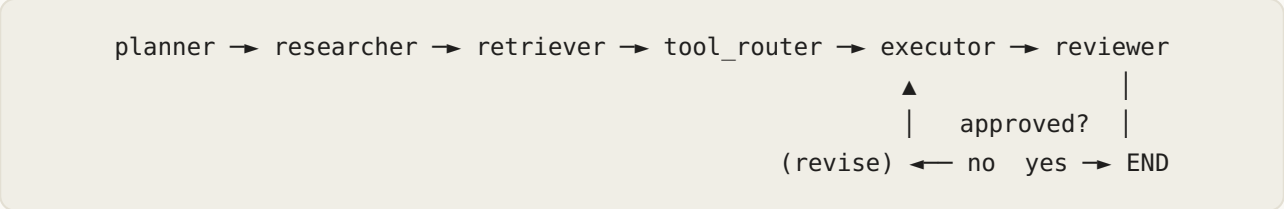
folder `project_multi_agent/`

This project combines all four pillars into a six-agent system, orchestrated by a small LangGraph-style graph runner. Each agent is a *node*; the shared state flows between them; every node is checkpointed; and agents share findings on a blackboard.

The six agents (each a graph node)

Agent	Job	Writes to state
Planner	Break the task into an ordered plan	`plan`
Researcher	Gather background facts	`research`
Retriever	Pull relevant documents	`documents`
Tool Router	Decide which tools to call	`tool_plan`
Executor	Run tools, assemble a draft	`tool_results, draft`
Reviewer	Approve, or send back to revise	`review, result`

The graph (note the loop)



The Reviewer is a **conditional edge**: if it approves, the run ends; if not, it loops back to the Executor to try again (with a hard cap to prevent infinite loops). Cyclic graphs like this are exactly what LangGraph is designed for.

The files and where each pillar lives

File	Pillar / role
`memory.py`	Pillar 1 — conversation memory (list)
`state.py`	Pillar 2 — shared AgentState (hash)
`checkpointer.py`	Pillar 3 — checkpoints (append-only list)
`shared_memory.py`	Pillar 4 — blackboard, locks, pub/sub

`agents.py`	The six agent node functions
`graph.py`	The orchestrator: runs nodes, routes, checkpoints, resumes
`main.py`	FastAPI endpoints
`run_cli.py`	CLI demo: full run + crash/resume

RUNS OFFLINE BY DEFAULT

With no `OPENAI_API_KEY`, agents use deterministic mock logic so the whole system runs with zero setup and zero cost. Set a real key and the same agents call real OpenAI.

CHAPTER 7

Running Everything

Step 1 — Start Redis

```
cd project_multi_agent
docker compose up -d          # or: redis-server --daemonize yes
redis-cli ping                # -> PONG
```

Step 2 — Run the four concepts

```
pip install redis
cd concepts
python 01_agent_memory.py       # remembers your name across turns
python 02_state_management.py   # planner and executor share state
python 03_checkpointing.py      # crashes, then resumes without repeating work
python 04_shared_memory.py      # blackboard + lock + pub/sub
```

Step 3 — Run the multi-agent system

```
cd project_multi_agent
pip install -r requirements.txt
python run_cli.py               # full run + a crash/resume demo

# Or as an API:
uvicorn main:app --reload
# open http://localhost:8000/docs
```

Step 4 — Watch it work

```
curl -X POST localhost:8000/run -H "Content-Type: application/json" \
     -d '{"task":"Explain how Redis powers AI agents"}'

# Then inspect the run:
curl localhost:8000/run/<run_id>/state    # the shared state
curl localhost:8000/run/<run_id>/board    # what each agent shared
curl localhost:8000/run/<run_id>/trace    # every checkpointed node
```

Mapping this to real LangGraph

We hand-built the graph so the mechanics are visible. In production you'd use LangGraph itself, which maps one-to-one onto what you learned here:

This project	Real LangGraph

node functions in `agents.py`	`graph.add_node(...)`
`LINEAR_NEXT` edges	`graph.add_edge(...)`
`_route_after_reviewer`	`graph.add_conditional_edges(...)`
`state.py` (Redis hash)	the typed `State` schema
`checkpointer.py`	`langgraph-checkpoint-redis`

Install `langgraph` + `langgraph-checkpoint-redis` to swap in the real framework once these ideas are second nature.

CHAPTER 8

Production Checklist & Takeaways

Habits that make agentic systems robust

- **Checkpoint after every node.** Long runs *will* be interrupted; make resume the default, not an afterthought.
- **Keep state in Redis, not in memory.** It survives crashes, deploys, and lets steps scale across machines.
- **Namespace every key** (`conv:`, `run:`, `thread:`, `lock:`) so pillars never collide.
- **Cap your loops.** A Reviewer that can send work back must have a hard revision limit, or it can loop forever.
- **Lock shared writes** with `SET NX PX` + a token; release atomically with Lua so you never delete someone else's lock.
- **Put a TTL on conversation memory** so idle sessions clean themselves up and Redis memory stays bounded.

The four pillars, one more time

Pillar	Structure	The one thing to remember
Memory	List	Replay recent turns; window + summarize
State	Hash	Single source of truth between nodes
Checkpoints	Append-only list	Resume instead of restart
Shared memory	Hash + Pub/Sub + locks	Cooperate without collisions

The five-second summary of Phase 6

LLMs forget everything between calls, so an agent's intelligence lives in what its framework stores and replays. Redis provides all of it: a *list* for conversation memory, a *hash* for the shared task state that flows between agents, an *append-only list* of checkpoints so long runs resume instead of restarting, and a *blackboard* (plus locks and pub/sub) so many specialist agents cooperate safely. Project 5 stacks all four into a six-agent graph you can run, crash, and resume.

Where you are now → Phase 7

You've built agents that remember, coordinate, and survive crashes. Phase 7 (Production Redis) is about running all of this reliably at scale: **clustering, Sentinel, persistence (RDB/AOF), monitoring, high availability, and disaster recovery** — turning your working systems into ones you can trust in production.